



Hello! How is it going?



zafar davlatov

fine maybe

good

i'm sick

i have no time for games

tired

work work work

Agenda

- Recall last session
- Discuss current topic
- Practice
- Quiz

Total Recall



What do you remember most from few previous workshops? (short answer)

phases

mvn

Maven related stuff

pom

architype

mvn --version

build lifecycle phases

pom.xml

What do you remember most from few previous workshops? (short answer)

pom

maven lifecycle

mvn

pom, dependencies

plugins

pluggin

Some maven related
topics

Maven project structure:

src/

---- main/

---- ---- java/

---- ---- resources/

---- test/

---- ---- java/

---- ---- resources/

target/

pom.xml

At which phase are integration tests executed?

15 ✓



verify

0 ✗

clean

0 ✗

compile

0 ✗

package

How does Maven resolve dependencies?

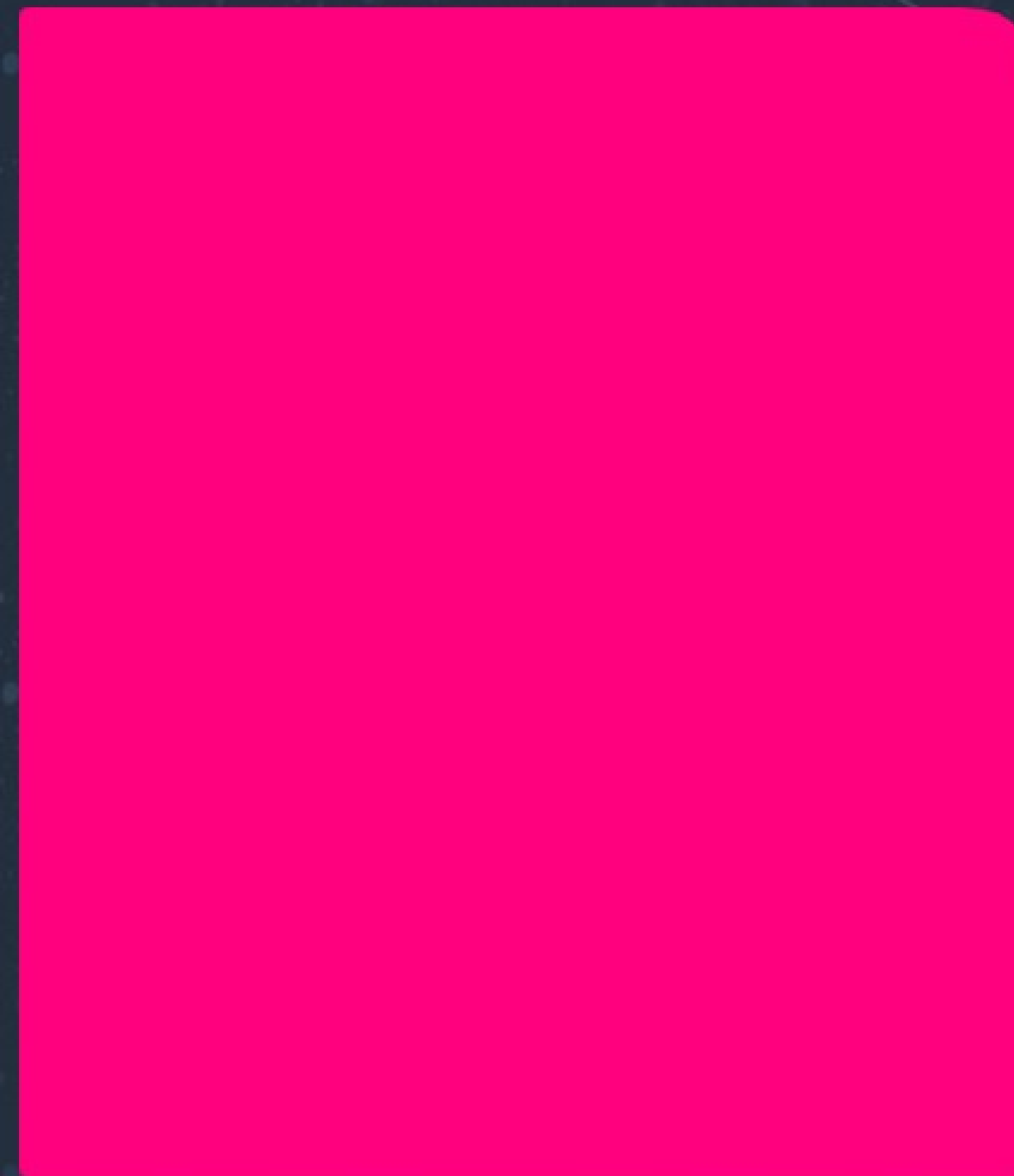


What is the default scope for Maven dependencies?



How do you exclude a dependency in Maven?

15 ✓



0 ✗

Remove it from ``$M2_HOME``
folder

0 ✗

Add an override for it

0 ✗

Manually delete the jar file

Use ``<exclusions>`` in POM



Next topic





Random fact

Scotland's national animal is the unicorn

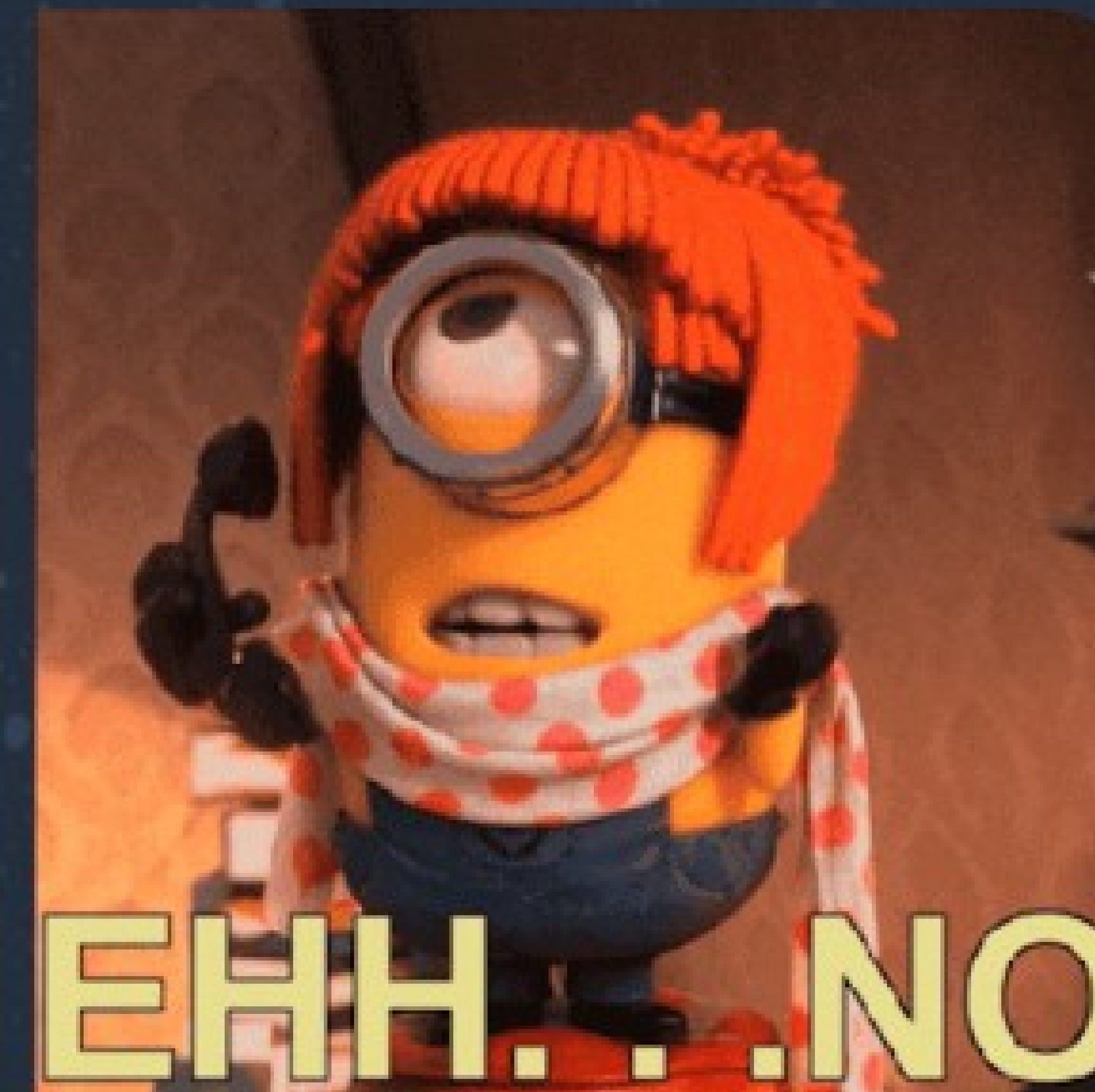
Have you looked through the materials?

10



yes

5



no

The Concept of Unit Tests

Types of Testing:

- Unit testing
- System testing
- Integration testing
- Acceptance testing
- Performance testing
- Regression testing
- Security testing
- Load testing
- End-to-end testing
- And more

ARRANGE -> ACT -> ASSERT

The main objectives of unit testing are:

- to isolate written code to test and determine if it works as intended
- to ensure that any other code changes do not ruin existing functionality, otherwise unit tests fail and highlight the issue
- developer can quickly understand the code, just looking at unit tests

Advantages of Unit Testing:

- Validate the smallest units of software
- Find bugs easily and early
- Save time and money
- Can force developers to write better and cleaner code

<https://junit.org/>

JUnit is a unit testing framework for the Java programming language.

TDD soul

JUnit follows the principle of "first testing, then coding," or setting up test data for a piece of code that can be tested first and then implemented—in other words, "test a little, code a little, start over." This approach increases the programmer's productivity and the stability of their code, reducing stress and the time spent on debugging.

Creating Unit Tests

1. code calculator in TDD style:
 - a. positive;
 - b. negative;
2. naming convention: `test{MethodName}_{positive|negative}()`
3. Idea debug;
4. debug calculate;
5. test coverage

Mock Objects

Mock Objects: Simulations of real objects that mimic actual dependencies, enabling:

- Predictable test execution
- Focus on specific component behavior
- Simplified testing process
- Improved test maintainability and reliability

Test Doubles (types)

Test doubles general name o simplified replacements for real dependencies, controlled during testing:

- Dummy - Placeholder passed as argument but never used
- Stub - Returns predefined values when called
- Fake - Working implementation with shortcuts (e.g., in-memory database)
- Spy - Partial mock recording calls while using real implementation
- Mock - Advanced simulation with configurable behavior and verification

Mock Objects Features

Behavior Configuration:

- Provide predefined responses/exceptions
- Create controlled, predictable test environment
- Validate specific scenarios

Interaction Verification:

- Check if methods were called
- Verify call order
- Count method invocations
- Validate passed arguments

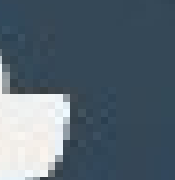
Java Mocking Libraries

- Mockito (most popular)
- JMock
- EasyMock



Quiz time

QUIZ



What is the purpose of a unit test?

0 ✗

Test entire application

0 ✓

Test individual units of
functionality

0 ✗

Test performance of code

0 ✗

Test the system as a whole

What is a unit in unit testing?

0 ✓

A small piece of code or function

0 ✗

An entire module

0 ✗

A database

0 ✗

A package

What is the main goal of unit testing?

0 ✗

Increase runtime performance

0 ✗

Test integrations

0 ✓

Identify bugs early in development

0 ✗

Optimize code performance

What is a mock object?

0 ✓

A simulated object mimicking
behavior

0 ✗

A real database

0 ✗

An actual implementation

0 ✗

A real API endpoint

When should you use a mock object?

0 ✗

Testing performance

0 ✓

When the real object is
unavailable or costly

0 ✗

Testing real database connections

0 ✗

During UI prototyping

A successful unit test is?

0 ✓

Isolated and repeatable

0 ✗

Used for real-time users only

0 ✗

Connected to real servers

0 ✗

Simulates full system functionality

What is required for effective unit testing?

0 ✗

Flexible deployment
scripts

0 ✗

Live servers for execution

0 ✓

Isolated test environments

0 ✗

Multi-threaded loops

What is the purpose of a verify function in a mock framework?

0 ✗

Return a string

0 ✗

Reset test cases

0 ✗

Create real-time queries

0 ✓

Confirm the expected method
was called

What is true about unit tests and mocking?

0 ✗

Unit tests skip testing mocks

0 ✗

Mocks slow down test
performance

0 ✓

Mocks help isolate the unit under
test

0 ✗

Unit tests require full data retrieval

Why are unit tests cost-effective?

0 

They skip over errors

0 

They identify bugs early in development

0 

Rely on static variables

0 

Eliminate all failures

Quiz leaderboard

No results yet

Top Quiz participants will be displayed here once there are results!

Q&A part

0 questions
0 upvotes

